INTERNSPARK

WEB APPLICATION SECURITY ASSESSMENT REPORT

Security assessment identifying SQL Injection and Cross-Site Scripting vulnerabilities through manual testing with Burp Suite, along with associated risks and mitigation techniques.

MAY 2026

Prepared by

SWAMANTAK ROY

Approved by

internSpark

Introduction:

Objective:

The objective of this assessment was to identify basic web application vulnerabilities in a safe demo environment through manual security testing techniques.

Target Information:

Parameter	Details
Application	OWASP Juice Shop
Testing Enviroment	Safe demo/test application
Testing Method	Manual web application testing

Scope:

The assessment was conducted according to the following requirements:

- Testing for:

1. Cross-Site Scripting (XSS)
2. SQL Injection

- Use of Burp Suite for intercepting and modifying HTTP requests
- Identification of vulnerabilities and associated risks
- Proposal of mitigation techniques

Tools Used:

Burp Suite

Used to:

- Intercept HTTP requests and responses
- Analyze application traffic
- Modify request parameters
- Test input validation weaknesses
- Perform manual vulnerability verification

Testing Methodology:

The security assessment followed a manual testing approach involving:

1. Intercepting requests using Burp Suite
2. Modifying application input parameters
3. Injecting test payloads
4. Observing application responses
5. Confirming successful exploitation

The testing focused on identifying vulnerabilities related to:

- Authentication mechanisms
- Input validation
- Client-side script execution

SQL injection Testing:

Test Objective:

The objective of this test was to determine whether the authentication mechanism was vulnerable to SQL Injection attacks through improper input validation.

Testing Environment:

The objective of this test was to determine whether the authentication mechanism was vulnerable to SQL Injection attacks through improper input validation.

Parameter	Details
Application	OWASP Juice Shop
Target URL	http://localhost:3000
Testing Tool	Burp Suite
Test Type	Manual SQL Injection Testing

Testing Procedure:

Step 1 — Intercept Login Request
The login request generated by the application was intercepted using Burp Suite Proxy.
The intercepted request contained:

- Email parameter
- Password parameter

Step 2 — Send Request to Repeater
The captured request was forwarded to the Burp Suite Repeater module for manual modification and testing.

Step 3 — Inject SQL Payload
The email parameter was modified with a SQL Injection payload designed to bypass authentication logic.
Payload Used

```
{
 "email": "' OR 1=1--",
 "password": "anything"
}
```

Step 4 — Submit Modified Request
The manipulated request was sent to the server through Burp Suite Repeater.

Step 5 — Analyze Server Response
The server returned a successful authentication response.
Response included:

- HTTP 200 OK
- Authentication token
- Administrator account details

SQL injection Findings:

Risk Level:
High

Description:
The login functionality did not properly validate user input, allowing SQL Injection through the authentication form.
The vulnerability enabled manipulation of backend query logic and resulted in successful authentication bypass.

Payload Used

```
{
  "email": "' OR 1=1--",
  "password": "anything"
}
```

Evidence:
The server responded with:
“HTTP/1.1 200 OK”
The response included:

- Valid authentication token
- Admin email: “admin@juice-sh.op”
- Role: admin

This confirms successful authentication bypass using SQL Injection.

Impact:
Successful exploitation may result in:

- Unauthorized administrator access
- Exposure of sensitive application data
- Full application compromise
- Privilege escalation
- Unauthorized modification of resources

Root Cause
The application dynamically processes user-controlled input without proper validation or secure query handling.

Mitigation Recommendations:

- Use parameterized queries (prepared statements)
- Avoid dynamic SQL query construction
- Validate and sanitize user input
- Implement secure authentication logic
- Apply least-privilege database permissions

XSS Testing:

Test Objective:

The objective of this test was to identify whether the application was vulnerable to Cross-Site Scripting attacks through improper handling of user-supplied input.

Testing Environment:

Parameter	Details
Application	OWASP Juice Shop
Target URL	http://localhost:3000
Testing Tool	Burp Suite
Test Type	Manual XSS Testing

Testing Procedure:

Step 1 — Identify Input Field
An injectable input field such as:

- Search bar
- Feedback form
- URL parameter

was identified for testing.

Step 2 — Inject XSS Payload
A malicious JavaScript payload was entered into the vulnerable input field.
Payload Used
“<img src=x onerror=alert(1)>”

Step 3 — Submit Payload
The payload was submitted to the application through the browser.

Step 4 — Observe Browser Behavior
The browser executed the injected JavaScript code and displayed a popup alert.

XSS Findings:

Risk Level:
High

Description:
The application allowed execution of malicious JavaScript code through unsanitized user input fields.
The injected payload executed successfully within the browser context.

Payload Used:
“<img src=x onerror=alert(1)>”

Evidence:
The browser displayed the popup message:
“localhost:3000 says 1"
This confirms successful script execution.

Impact:
Successful XSS exploitation may allow:

- Session hijacking
- Cookie theft
- Malicious script execution
- Unauthorized actions on behalf of users
- Client-side data theft
- Redirection to malicious websites

Root Cause
The application renders user-controlled input without proper sanitization or output encoding.

Mitigation Recommendations

- Sanitize all user inputs
- Implement output encoding
- Use Content Security Policy (CSP)
- Avoid rendering raw user input in HTML
- Validate and filter special characters

Overall Security Assessment:

The assessment identified two major web application vulnerabilities:

- SQL Injection
- Cross-Site Scripting (XSS)

Both vulnerabilities are classified as high risk due to their ability to compromise:

- Authentication security
- User data confidentiality
- Session integrity
- Application functionality

The findings indicate insufficient input validation and insecure handling of user-controlled data.

General Security Recommendations:

Input Validation

- Validate all user inputs on both client and server sides.
- Reject unexpected or malformed input.

Secure Coding Practices

- Follow secure coding standards during development.
- Avoid insecure query construction and unsafe rendering methods.

Output Encoding

- Encode user-generated content before displaying it in the browser.

Security Headers

- Configure Content Security Policy (CSP).
- Use secure HTTP response headers.

Regular Security Testing

- Conduct periodic vulnerability assessments and penetration testing.
- Integrate security testing into the development lifecycle.

General Recommendations:

Secure Input Handling

- Validate all user-supplied data on both client and server sides.
- Reject malformed or unexpected input.

Secure Coding Practices

- Follow secure development standards.
- Avoid insecure query handling and unsafe rendering methods.

Security Testing

- Perform regular penetration testing.
- Conduct periodic vulnerability assessments.
- Integrate security testing into the software development lifecycle.

Conclusion:

The web application security assessment successfully identified multiple high-risk vulnerabilities in the test environment. The SQL Injection vulnerability enabled authentication bypass, while the XSS vulnerability demonstrated arbitrary JavaScript execution.

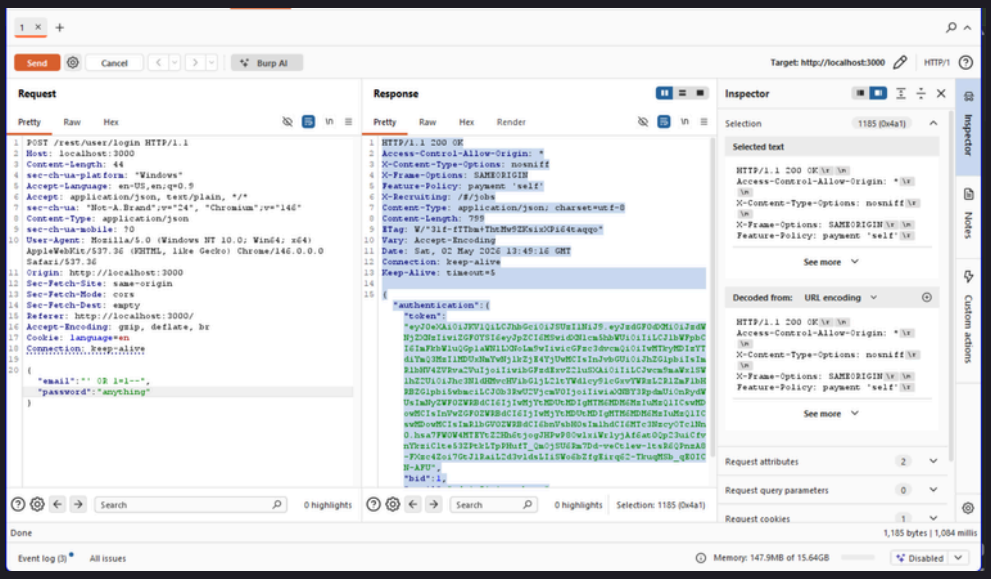
Applying the recommended mitigation techniques would significantly improve the security posture of the application and reduce the risk of exploitation.

Appendix:

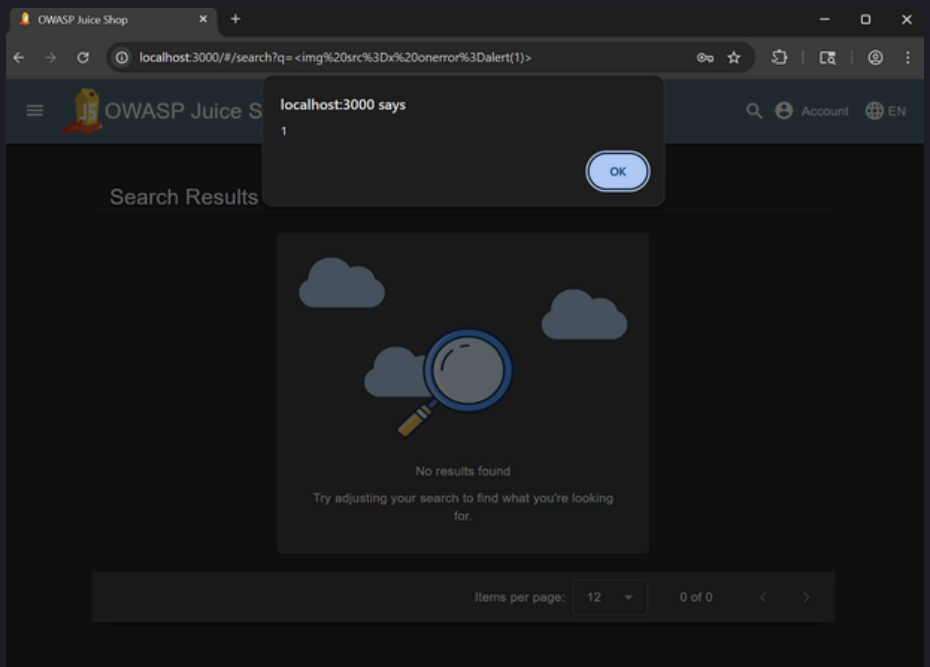
Tools Used:

Burp Suite

Screenshots:



SQL INJECTION



cross site scripting XSS